The Self-Healing Web

Designing Resilient, Typo-Proof URLs

Dominic Hollis

EuroPython 2026

404: a dead end for everyone

- Users hit a wall over a poorly formatted URL - a typo, wrong case, a missing word
- Developers (You) lose the visit: a bounce, a lost referral, a support ticket
- The web forgives fuzzy input everywhere else (search boxes, autocomplete) - except the address bar

Defining "self-healing"

- Moving on from **strict routing** to a **resilient, user-friendly** one
- The **unique identifier** (id / primary key / slug suffix) is the *source of truth*
- The **human-readable** portion (slug text, casing, word order) is allowed to be flexible - or even missing
- Goal: resolve the user's *intent*, not just match a string

Who is using self-healing URLs?

You've probably used one already

- `github.com/user/renamed-repo` → old repo name still redirects to the new one
- `stackoverflow.com/questions/12345/anything-here` → the slug text is decorative, the id is truth
- `amazon.de/dp/B0FH23F6SB/something` → same pattern: the product id (after `/dp/`) is the truth, the rest is cosmetic
- Wikipedia: case-insensitive first letter, redirects for common aliases/misspellings

The resilience mindset

- Old mindset: **"I can't find that"** → 404, dead end, blame the user
- New mindset: **"I know what you meant - let me find and fix it for you"**
- A redirect costs one extra request; a lost user can cost a customer

What should (and shouldn't) be healed

Fair game: typos, casing, word order, missing slug words, renamed-but-tracked resources

Off limits: auth-gated routes, destructive actions, admin paths - anywhere "close enough" is a security risk

Techniques for implementing self-healing URLs

+ Using flask-selfheal

Technique 1: Partial string matching

- Cheapest first: exact match, then substring / `LIKE`-style matches
- `WHERE slug LIKE '%term%'` - fast, index-friendly, easy to reason about
- Good for: missing or extra words, partial slugs, reordered fragments
- Limit: doesn't help with genuine typos or misspellings

Technique 2: Fuzzy matching

- Handles typos & near-misses that substring matching can't
- Similarity scoring (e.g. Python's `difflib.SequenceMatcher`) between the requested path and known candidates
- A configurable **cutoff** threshold decides "close enough" vs. "not a match"
- Trade-off: more forgiving, but more expensive and more prone to false positives

Technique 3: Hybrid, chained resolution

Real apps need more than one trick - chain cheap-to-expensive strategies.

flask-selfheal's `DatabaseResolver` tries, in order:

1. Exact match
2. SQL `LIKE` matching
3. Normalized matching (`0→o` , `1→l` , ...)
4. Word-based matching
5. Partial matching
6. Fuzzy matching *(last resort)*

flask-selfheal, under the hood

- `SelfHeal(app, resolvers=[...])` - one Flask extension, a *list* of resolvers
- Resolvers are **chainable** - each is tried in order until one resolves the path
- Built-ins: `FlaskRoutesResolver`, `AliasMappingResolver`, `FuzzyMappingResolver`, `DatabaseResolver`
- Plug in only what you need; combine them for defense in depth

Case study: fixing route typos

```
from flask_selfheal import SelfHeal
from flask_selfheal.resolvers import FlaskRoutesResolver

@app.route("/home")
def home(): ...

@app.route("/about")
def about(): ...

@app.route("/contact")
def contact(): ...

SelfHeal(app, resolvers=[FlaskRoutesResolver()])
```

```
/hme    -->  /home
/abot   -->  /about
/contat -->  /contact
```

Fuzzy-matches against your *already registered* Flask routes - zero extra config.

Case study: chaining resolvers

```
from flask_selfheal.resolvers import AliasMappingResolver, FuzzyMappingResolver

resolvers = [
    AliasMappingResolver({"old-path": "new-path"}),
    FuzzyMappingResolver(["home", "new-path"]),
]

SelfHeal(app, resolvers=resolvers)
```

```
/old-path -->  /new-path   (alias)
/hme      -->  /home       (fuzzy)
/new-pth  -->  /new-path   (fuzzy)
```

Explicit aliases for known renames; fuzzy matching as the safety net.

Case study: healing slugs against a database

```python
from flask_selfheal.resolvers import DatabaseResolver

resolver = DatabaseResolver(
    Articles,
    slug_field="slug",
    fuzzy_cutoff=0.7,
    enable_word_matching=True,
    enable_partial_matching=True,
    custom_normalizers={"0": "o", "ph": "f"},
)

SelfHeal(app, resolvers=[resolver], redirect_pattern="/articles/{slug}")
```

```
/articles/hell-wrl  -->  /articles/hello-world-1234567
```

Same tuning knobs that power the last three techniques, exposed as config.

Live demo

1: A typo in a URL segment
2: A missing / reordered slug word
3: Case sensitivity
4: A database-backed slug, healed end-to-end

What happens on a request?

404 raised → Resolver 1: exact → Resolver 2: fuzzy → Resolver 3: DB match

match found 301 / 302 redirect **no match anywhere** real 404

Resolvers are tried in the order you configure them - first match wins.

Limitations & trade-offs

- Fuzzy matching can produce false positives - "close enough" isn't always "correct"
- DB fallback strategies (word / partial / fuzzy) cost more than an indexed exact lookup
- Redirect chains can confuse caching/SEO if not paired with proper 301s
- Healing should never extend to security-sensitive or destructive routes

Recap

- 404s are a dead end - self-healing URLs turn them into a redirect
- Treat the identifier as truth, keep the human-readable part flexible
- Chain cheap → expensive resolution strategies (string match → fuzzy → DB)
- flask-selfheal packages this up as composable resolvers for Flask

🔗 github.com/GovernmentPlates/flask-selfheal

📦 pypi.org/project/flask-selfheal